ACCESCO LIVING PRIVATE LIMITED

Warehouse / Store-Facing Replenishment Engine

Step-by-step guide to forecasting demand and recommending when and how much to replenish

Plain-language, step-by-step implementation manual
No code snippets; clear gates, owners and acceptance checks

Document ID	ACC-REPL-WH-IMPL-02
Version	Revision 2.0 - plain-language edition
Status	Implementation proposal - requires system-owner approval
Prepared for	Data & Analytics, Engineering, Operations and system owners

Important: this is a build guide, not evidence that the proposed APIs, data fields, systems or model performance already exist.

Contents

- 1. Executive summary and scope
- 2. Current facts, assumptions and decisions to confirm
- 3. The improved architecture, explained layer by layer
- 4. Responsibilities of OMS, IMS, WMS and TMS
- 5. Data checklist by source and processing layer
- 6. Technology and people needed
- 7. Step-by-step build plan from an empty workspace
- 8. How the forecast and stock recommendation are chosen
- 9. Worked example: one SKU at one store
- 10. Testing plan and acceptance checks
- 11. Pilot, deployment, rollback and ongoing support
- 12. Monitoring, security and business measures
- 13. Main risks and decisions required
- 14. Final go-live checklist
- References and supporting documents

1. Executive summary and scope

The warehouse/store-facing engine helps operations answer three practical questions for every product at every replenishment location: Is replenishment needed? What quantity should be proposed? How soon should staff act? It does this by estimating future demand, then combining that estimate with actual stock, reserved stock, incoming supply and operating limits. The model makes a recommendation; an authorised person approves the stock action.

This is not a single model that directly invents an order quantity. It is two connected parts: (1) a demand forecast that estimates likely sales over future periods; and (2) a clear set of inventory rules that turns the forecast into a recommendation while respecting pack sizes, minimum order amounts, expiry and storage limits.

MVP boundary: Start with recommendations for selected SKUs and one or a few pilot stores / warehouses. No automatic purchase orders, stock transfers or inventory writes. Actual actions remain in the existing system that owns them.

What success looks like

- Every recommendation can be traced to its source stock position, forecast period, lead time and business rules.
- Operations can approve, reject or adjust a recommendation and record why.
- The team can measure whether recommended actions reduce stockouts without creating avoidable overstock and expiry.
- When the data are incomplete or stale, the engine says so and directs the case to manual review.
- Forecasting performance is demonstrated on Accesco history, not claimed from a public benchmark or synthetic test data.

2. Current facts, assumptions and decisions to confirm

Item	Current understanding	Required confirmation
Confirmed starting point	SKU/product catalogue and inventory data exist in PostgreSQL. Databricks plus PostgreSQL and APIs is the intended platform.	Identify exact database, tables, owners, access method and refresh constraints.
Sales history	Historical orders/sales are limited or not yet verified.	Confirm whether fulfilled item lines, timestamps, cancellations, substitutions and returns can be retrieved.
Human approval	Staff approval is required before a stock action.	Name the role that may approve and whether a second approval is needed for large/high-cost orders.
System names	Working split: OMS owns order lifecycle; inventory IMS owns catalogue and stock; WMS owns physical warehouse operations; TMS owns transport/delivery records.	Resolve the naming conflict where “IMS” also means an integrated governance framework in some Accesco documents.
Supplier and purchase orders	Supplier/PO workflow may be SRM, another procurement tool, or a manual process.	Confirm if an SRM/procurement system exists. Do not assume the engine can create purchase orders through IMS.
Transport	TMS may provide inbound truck ETA and/or last-mile delivery events.	Confirm whether the same TMS covers DC-to-store freight as well as customer delivery.
Nodes and hierarchy	A central warehouse / distribution centre may replenish multiple dark stores.	Confirm node IDs, node types, permitted source-to-destination routes and who owns transfer rules.
Forecast tool	TimesFM 3.0 is an evaluation candidate from the earlier architecture.	Its self-hosted public weights have a non-commercial / non-production restriction. Approve a permitted commercial route or compare a different model before production.
Targets and dates	Pilot sites, budget, assigned team and start date are not confirmed.	Agree these at project kickoff; do not invent targets or owners.

Critical model gate: A product catalogue and current stock alone cannot teach a model how demand changes over time. Until usable, timestamped sales and availability history is available, build the data flow and test the inventory rules, but do not claim that a trained demand model is ready.

3. The improved architecture, explained layer by layer

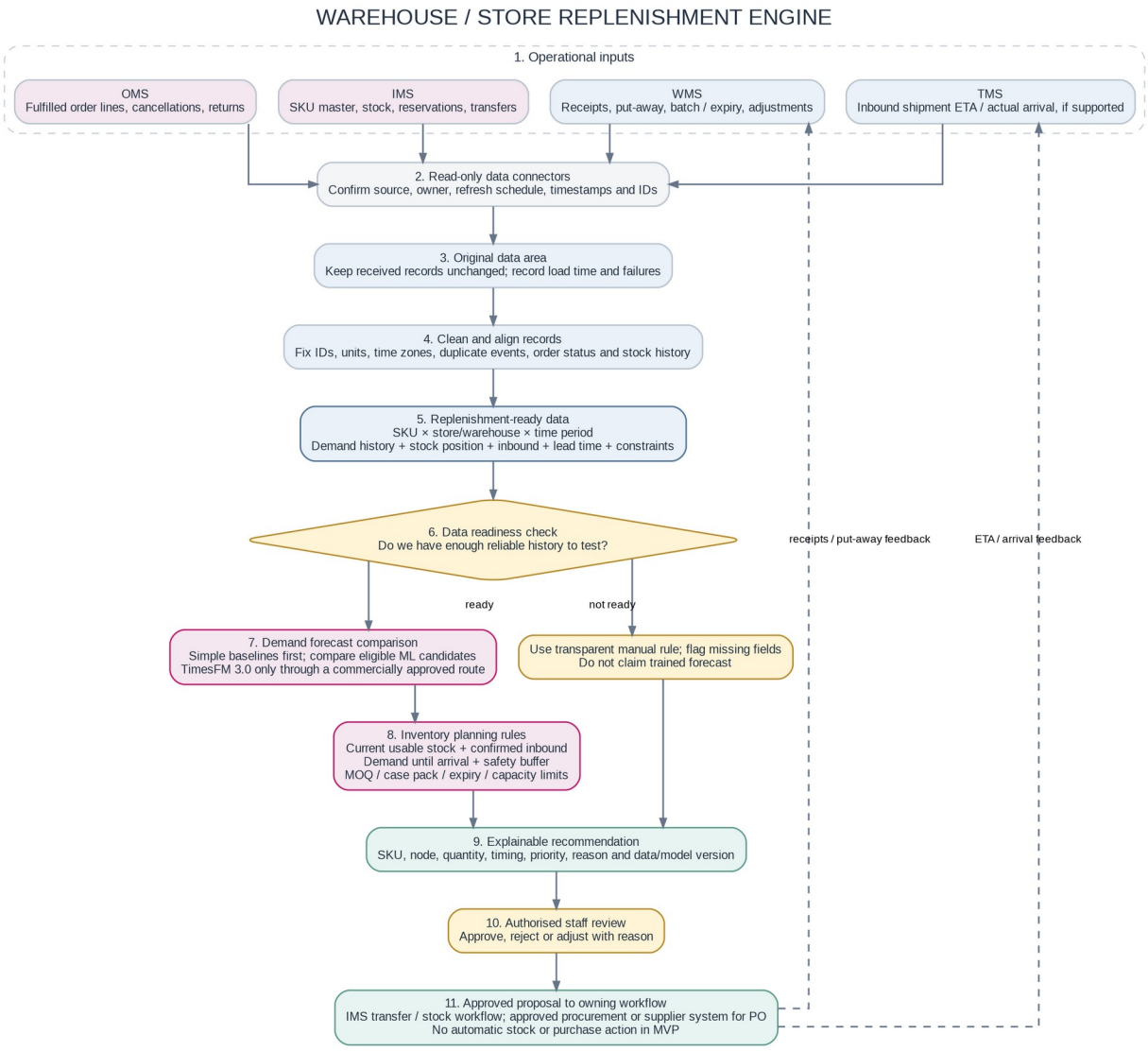


Figure 1. Warehouse/store engine: source systems, data preparation, forecast, stock rules and staff approval.

The architecture separates the records used to run the business from the records used for analysis. OMS, IMS, WMS and TMS remain responsible for their own transactions. Databricks prepares reliable history and runs forecasting; a recommendation record is sent to a review screen or approved report. Only after staff approval does the owning operational workflow receive an action request.

Layer	Plain-language job	What it stores or produces	Important rule
1. Source systems	Collect facts from systems already used by the business.	Orders actually fulfilled; stock and reservations; receipts and put-away; confirmed inbound shipments and arrival times.	Never calculate stock by guessing from order counts.

Layer	Plain-language job	What it stores or produces	Important rule
2. Connection layer	Move data through approved APIs or scheduled extracts.	Source name, read time, time range, row counts, failures and source event IDs.	Start with scheduled extracts if real-time events are not ready.
3. Original-data area	Keep an unchanged copy of what was received.	Source snapshots and load history.	Do not overwrite source evidence with cleaned versions.
4. Cleaned-data area	Make names, times, quantities and statuses consistent.	One recognised SKU, one recognised location, consistent units and deduplicated events.	Rejected or unclear records must be listed for fixing, not silently dropped.
5. Business-ready data	Prepare one time series per SKU and node, with inventory facts alongside.	Demand by agreed time bucket; availability; incoming supply; lead-time and product constraints.	Use hourly data only when the source history supports hourly accuracy; otherwise start daily.
6. Forecast comparison	Test simple methods and eligible ML candidates on past periods.	Forecasts for the same products, nodes and future periods, along with error measures.	Choose on Accesco backtests and operational impact, not model reputation.
7. Inventory planning	Turn the forecast and current stock into a proposed quantity and timing.	Inventory position, expected demand until arrival, target stock, pack rounding and constraint checks.	This is business logic with explicit rules, not another black-box model.
8. Recommendation and review	Show the result and why it was suggested.	SKU, store, quantity, urgency, expected stockout window, reasons, versions and staff decision.	No recommendation may directly adjust stock in the MVP.
9. Feedback	Use what actually happened to improve the next run.	Actual receipts, delivered quantities, delays, stock adjustments, overrides and final stock.	Use confirmed outcome events; do not count a draft as a completed replenishment.

4. Responsibilities of OMS, IMS, WMS and TMS

System	What the engine needs from it	What the engine may return	What it must not take over
OMS - Order Management System	Order line, SKU, quantity, order status, fulfilment status, cancellation, return, substitution, node and final fulfilment time.	Forecasts and stock recommendations may be displayed in a shared admin view, but OMS is not the owner of physical inventory.	Do not change order states or infer fulfilled sales from placed orders.
Inventory IMS	SKU catalogue, on-hand and usable stock, reservations, blocked/expired stock, inventory adjustments, stock transfers and location mapping.	An approved stock transfer or inventory workflow request, only through a documented and permissioned interface.	Do not write stock balances or create a second inventory ledger.
WMS - Warehouse Management System	Receiving, actual quantities received, quality holds, batch and expiry, put-away completion, pick/pack adjustments and dispatch events.	Approved inbound expectation or transfer work request only if WMS supports that action and the contract is agreed.	Do not invent receipt, put-away or bin status from planned quantities.
TMS - Transport Management System	Confirmed shipment, planned arrival, ETA changes and actual arrival time for inbound replenishment; customer delivery events where useful.	An approved shipment-planning request only if inbound transport is in scope and TMS exposes the needed capability.	Do not assume a TMS built for last-mile riders also handles line-haul freight.
SRM / procurement tool (if present)	Approved suppliers, supplier lead times, minimum order quantities, pack rules, contracted availability and purchase-order status.	Draft purchase proposal or approved request through the supplier/procurement owner.	If this system does not exist, show a manual purchase proposal rather than inventing an integration.

Integration principle: Use one authoritative owner for each operational fact. The replenishment engine should read facts, calculate a proposal and record its reasoning. OMS/IMS/WMS/TMS remain the owners of the order, stock, warehouse and transport status.

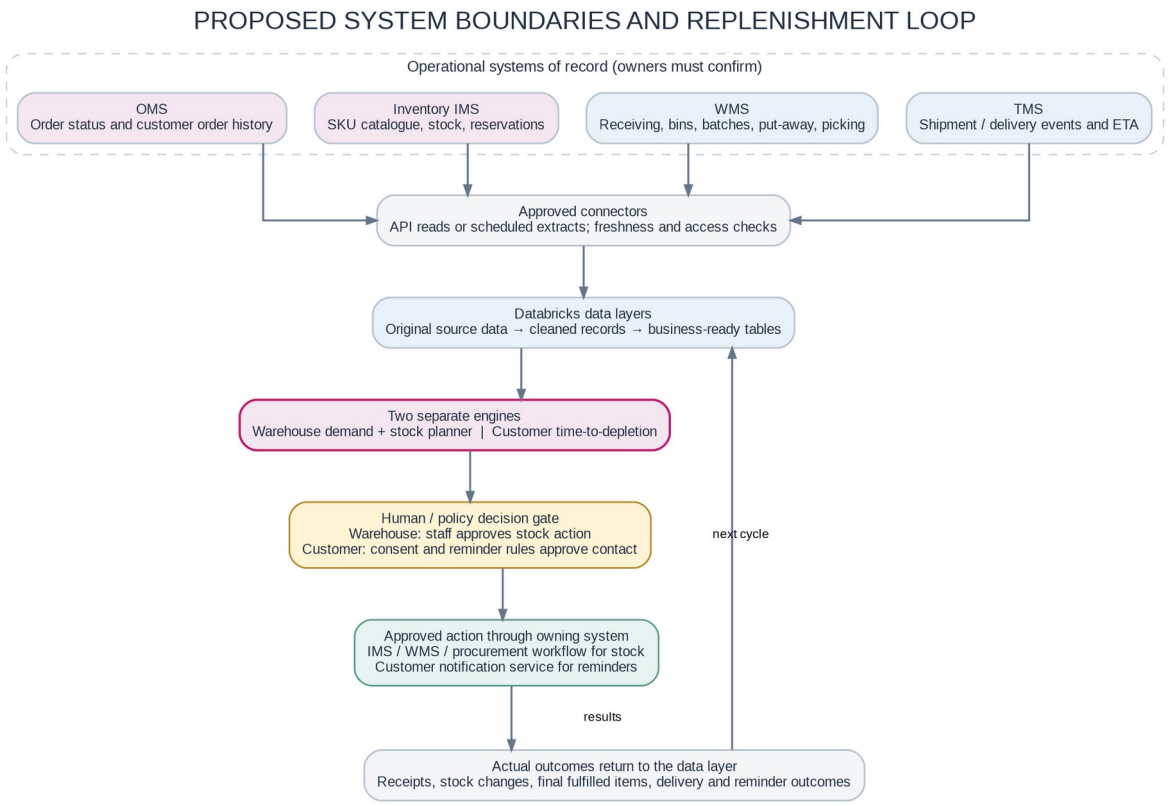


Figure 2. Shared operating boundary proposed for both replenishment engines. Exact API names and event formats are to be agreed by the system owners.

5. Data checklist by source and processing layer

Use this as the first data-audit checklist. A field marked “required before trained forecast” blocks model training/evaluation if it cannot be reconstructed reliably. “Required before operational recommendation” blocks a recommendation action if the current value is missing or stale.

Data group	Required information	Why it is needed	Priority / fallback
Product/SKU master	Stable SKU ID; product name; category; active/inactive state; base unit; pack/case size; unit conversion; supplier mapping if available; shelf life and expiry rules for applicable goods.	Prevents products with different pack sizes or identifiers being combined.	SKU ID and unit are required immediately. Missing pack/expiry rules trigger manual review for affected items.
Location / node map	Stable location ID; node type such as distribution centre or dark store; timezone; active state; source/destination relationship.	Forecasts must be for the correct location and transfers must follow permitted routes.	Required before combining inventory or demand across locations.
Sales / order-line history	Order ID; line ID; SKU; node; ordered quantity; fulfilled quantity; order status; cancellation/return/substitution; placed time; fulfilled time.	Measures actual sales rather than requests that were later cancelled or unavailable.	Required for demand model backtesting. Prefer fulfilled quantities and time.
Stock history	On-hand; reserved/committed; unavailable/quality-held; expiry-blocked; available; adjustments; inventory movement reason; effective timestamp.	Shows what stock was truly sellable and when an item became unavailable.	Required for inventory planning; history is needed to identify sales suppressed by stockouts.
Availability / stockout	Periods a SKU could be sold, unavailable periods, start/end time and reason if known.	Zero sales while out of stock is not evidence of zero customer demand.	Required for honest demand evaluation; if missing, flag likely censored demand and report the limitation.

Data group	Required information	Why it is needed	Priority / fallback
Purchase and receipt records	Purchase request/PO ID if available; order time; supplier; SKU; ordered amount; promised arrival; actual arrival; quantity received; partial receipts; rejected quantity.	Measures lead time and whether incoming quantity can be counted as supply.	Confirmed inbound must be distinguished from unapproved or speculative supply.
WMS receiving and quality	Goods receipt, accepted/rejected quantity, damaged/quality hold, batch, expiry, put-away completion and reason for adjustment.	Only usable stock should count. Perishables must not be replenished beyond likely sell-through or expiry.	Required for strict perishable/expiry logic; until reliable, mark those recommendations for review.
TMS shipment timeline	Shipment ID, source and destination node, linked PO/transfer, departure time, current ETA, delay reason and actual arrival.	Makes incoming supply timing more realistic and helps measure actual lead time.	Required to use live ETA; otherwise use confirmed lead time plus an approved buffer.
Business policies	Target service level or allowed stockout risk; review frequency; reorder rules; supplier minimum order; case multiple; storage capacity; budget limit; expiry and shelf-life guardrails.	Controls the trade-off between stockout risk and excess stock.	Operations must approve values. Do not manufacture policy values inside the model.
Promotions / calendar (later if unavailable)	Known promotion dates, price changes, holidays, weekdays, hours and major local demand events.	Explains demand spikes that history alone may not capture.	Start with calendar patterns available; promotions are useful but not a blocker for a rule-based prototype.

6. Technology and people needed

Need	Recommended starting choice	Why / when to reconsider
Operational database	Existing PostgreSQL	Continue to use it for operational data and agreed recommendation records. Avoid duplicating stock/order truth.
Analytics and history	Databricks with managed lakehouse tables	Keeps original extracts, cleaned history, forecasting data and results in a controlled place.
Data preparation	Databricks jobs and Spark/SQL transformations maintained by the data team	Repeatable data cleaning and table refresh; not manual spreadsheet processing every day.
Forecast models	Simple baseline first; compare LightGBM/XGBoost when useful history exists. TimesFM 3.0 remains an option only through a commercially permitted route.	A good model must beat simpler options on local data. Do not self-host public TimesFM 3.0 weights in a commercial production system without rights.
Experiment and model record	MLflow / governed Databricks model registry if available in the workspace	Records which data, model settings and evaluation results produced each candidate.
Scheduled operations	Databricks job scheduler / Workflows available in the workspace	Runs data refresh, checks, forecasts and reports in a repeatable schedule.
Staff review	IMS/admin panel screen if supported; otherwise a controlled report/export for the pilot	Start without waiting for a complex new UI, but log every decision and override.
Monitoring	Databricks job and data-quality alerts plus an agreed operations dashboard	Shows stale data, failed jobs, forecast changes and business outcome trends.
Security	Role-based access, protected credentials, approved network route, separate development/test/production access	Keeps customer/order/stock information and system write privileges limited.
Optional components	A message bus, online prediction endpoint or additional cache only if required by volume/latency and supported in the current platform.	Do not add infrastructure that creates maintenance burden before the MVP needs it.

Roles needed (people not yet assigned)

Role	Responsibility	Status
Business / Operations owner	Defines acceptable stockout risk, review rules, expiry guardrails and approval policy.	Yet to be assigned
Data engineer	Maps PostgreSQL/APIs, creates reliable history and data checks.	Yet to be assigned
ML / analytics lead	Builds the comparison, evaluates forecast quality and explains limitations.	Yet to be assigned
Backend / integration engineer	Agrees and implements read APIs and approved hand-off interfaces.	Yet to be assigned
IMS/WMS/OMS/TMS system owners	Confirm field meanings, API/event contracts, test environment and system-of-record boundaries.	Yet to be assigned
Security / privacy reviewer	Approves access, credentials, retention and audit requirements.	Yet to be assigned

7. Step-by-step build plan from an empty workspace

Follow the steps in order. The first release should prove that data and inventory rules work before making any production action. Each step has an output to review before moving on.

Step 1. Approve the operating definition

Purpose: Avoid building the wrong kind of replenishment.

What to do:

- Name the pilot purpose: central warehouse to dark store, supplier to warehouse, dark store stock top-up, or a specific combination.
- Confirm which node types exist and whether each SKU can be supplied by one or multiple source nodes.
- Agree the time unit for the first forecast (hourly only when data support it; otherwise daily).
- Name the approving role and record what that role may approve, reject or change.
- Confirm who owns supplier orders or transfer requests; keep this as a manual step if the purchasing system is not chosen.

Expected output: A short signed decision sheet covering scope, node hierarchy, timing, owner and approval authority.

Do not move ahead until: No data connection or stock action is allowed until ownership and pilot scope are understood.

Step 2. Inventory all source data

Purpose: Find out what can genuinely be measured before selecting a model.

What to do:

- Ask the PostgreSQL owner for the catalogue, stock, reservations and inventory movement locations.
- Ask OMS owners for item-level orders and final fulfilment status, including cancellations, returns and substitutions.
- Ask WMS owners for receiving, rejected quantities, put-away, batch/expiry and inventory adjustment history.
- Ask TMS owners whether inbound shipment ETAs and actual arrivals can be linked to an order or transfer.
- For each field, record owner, refresh frequency, time zone, meaning, historical coverage and known gaps.

Expected output: A source map and data dictionary with each field marked available, partial, missing or unclear.

Do not move ahead until: Do not proceed to training until the team knows whether timestamps and sales quantities can be trusted.

Step 3. Create secure connections

Purpose: Move source facts without giving the model permission to alter them.

What to do:

- Create a development workspace and a separate place for test results.
- Use read-only access to operational sources for the initial ingestion.
- Agree API access or scheduled extracts with source-system owners; record failure and retry behavior.
- Capture the extraction time, source record ID, source update time and load result for each import.
- Keep credentials in the approved secret store; never place passwords in spreadsheets or notebooks.

Expected output: A repeatable, read-only data load that can be rerun and audited.

Do not move ahead until: A failed or partial import must be visible; it must not be treated as a complete stock snapshot.

Step 4. Build the original-data area

Purpose: Keep a record of exactly what was received.

What to do:

- Store source snapshots separately by source and date/time received.
- Keep source identifiers and original values even when they need correction later.
- Record row counts, latest timestamps, rejected records and duplicate counts.
- Agree how long operational extracts are kept and who can read them.

Expected output: An auditable raw-data area with a load-history report.

Do not move ahead until: If a result is questioned, the team must be able to trace it to the source version.

Step 5. Clean and standardise the data

Purpose: Make the same SKU and location mean the same thing across systems.

What to do:

- Map each source SKU to the approved master SKU; maintain a controlled mapping for legacy IDs or pack variants.
- Convert quantities into a recognised base unit without confusing individual items, packs, cases, litres or kilograms.
- Standardise event times and time zones; distinguish when an order was placed, fulfilled, received and put away.
- Remove duplicate events using source IDs and approved duplicate rules.
- Separate completed/fulfilled sales from orders that were cancelled, refunded, returned or substituted.
- Create a data quality report listing missing IDs, invalid quantities, negative stock, impossible dates and conflicting statuses.

Expected output: Cleaned, consistent tables plus a list of records that require owner correction.

Do not move ahead until: Do not quietly replace missing lead times, pack sizes or negative stock with guessed values.

Step 6. Rebuild historical demand honestly

Purpose: Build a reliable history of what customers actually bought, not what they tried to buy.

What to do:

- Combine fulfilled order lines with their correct SKU and fulfilment location.
- Aggregate actual fulfilled quantities into the agreed time bucket for each SKU and node.
- Use availability and inventory history to mark periods when a product could not be purchased.
- Flag demand periods affected by stockouts, partial fulfilment or missing order records.
- Check that day totals match OMS/WMS reports for selected sample dates and locations.
- Where history is too short or incomplete, keep a smaller pilot scope and record the limitation.

Expected output: A SKU-by-node demand history with reconciliation evidence.

Do not move ahead until: No trained forecast claim until the data can be compared with actual future periods that were not used to build it.

Step 7. Build the first simple forecasts

Purpose: Create a fair comparison before choosing a complex model.

What to do:

- Create a recent-average forecast.
- Create a same-weekday or same-hour-last-week forecast where the data contains that pattern.
- Where suitable, add a simple smoothing method that gives more weight to recent demand.
- Use the exact same forecast dates and target horizons for every candidate.
- Run a chronological backtest: use the past to predict a later period, then compare prediction with what actually happened.
- Report results by store, category, fast/slow selling products and stockout exposure, not just one overall score.

Expected output: A baseline comparison report that Operations can understand.

Do not move ahead until: If data history is too short to create a fair holdout, report that the result is inconclusive.

Step 8. Evaluate the advanced forecast candidate

Purpose: Choose a model based on local evidence and legal permission.

What to do:

- If a candidate can be trained from the available data, test LightGBM or XGBoost with useful past-sales and calendar information.
- If TimesFM 3.0 is considered, first confirm a commercially authorised route. Publicly downloadable 3.0 weights must not be assumed suitable for commercial production.
- Ask whether the candidate can use the available time periods and covariates; do not invent missing hourly values or promotions.
- Compare the candidate to simple baselines across multiple historical cut-off dates.
- Inspect large errors and consistent over-forecasting or under-forecasting in specific products and stores.
- Record the winning candidate, rejected candidates, data cut-off and reason for selection.

Expected output: An evaluation note and a chosen candidate only if it proves useful.

Do not move ahead until: A more complicated model is not promoted simply because it is newer or more advanced.

Step 9. Define the inventory rules with Operations

Purpose: Turn future demand into a stock proposal that obeys the business.

What to do:

- Agree how to calculate sellable stock: start from usable on-hand and deduct reservations, blocked items and expired goods.
- Count only confirmed incoming quantities expected before they are needed; avoid counting the same shipment twice.
- Estimate expected demand during the time needed for supply to arrive.
- Agree a safety buffer for unexpected demand and delays; initially it may be a documented policy rather than a learned value.
- Agree the desired stock cover after arrival and the review interval.
- Apply minimum order amount, pack/case multiple, available storage, shelf life and any spending limit.
- If rules conflict (for example, minimum supplier order would exceed safe shelf-life limits), stop and flag staff review.

Expected output: A written inventory policy, with examples for normal, urgent and constrained products.

Do not move ahead until: Inventory policy values and safety buffers must be approved rather than quietly set by the ML team.

Step 10. Create the recommendation record

Purpose: Make the result understandable and traceable.

What to do:

- For each SKU and node, record the current stock cut-off time and its source.
- Show forecast demand over the relevant period and the expected time until stock becomes critical.
- Show target stock, proposed quantity, pack rounding and any capacity/expiry restriction.
- Give a short reason such as high recent demand, short lead time remaining, confirmed inbound missing, or capacity conflict.
- Attach the forecast version, data version, calculation time and recommendation expiry time.
- Use clear statuses such as needs review, approved, rejected, expired or superseded.

Expected output: A reviewable recommendation list that can be reproduced later.

Do not move ahead until: A recommendation must be discarded or refreshed if its stock snapshot has become stale.

Step 11. Connect the recommendation to the admin workflow

Purpose: Allow staff to review without bypassing the owner system.

What to do:

- First use a controlled report/export or a simple review screen if the permanent panel is not ready.
- Show approve, reject and adjust actions with a mandatory reason for overrides.
- For stock transfers, call the approved IMS/WMS workflow only after its owner confirms the contract.
- For supplier purchases, route the proposal through the approved purchasing/SRM process; otherwise provide a purchase request for manual handling.
- Keep an idempotent reference so retrying a submission does not create a duplicate task.

Expected output: A tested staff approval path with a complete audit trail.

Do not move ahead until: The initial release must not directly alter stock, create an order or dispatch a transfer without an approved workflow.

Step 12. Test all connections and failure cases

Purpose: Prove the process works when systems are slow or data are wrong.

What to do:

- Test correct SKU and location mapping, unit conversions, duplicates and missing values.
- Test no stock, negative/held stock, incoming supply arriving after the predicted stockout and a delayed truck.
- Test expiry and storage-capacity constraints, minimum order, pack rounding and conflicting constraints.
- Test API timeout, duplicate submission, partial load, stale inventory, missing ETA and a rejected approval.
- Confirm the engine fails safely: it creates a visible exception and never makes up a quantity or status.

Expected output: A signed test record showing expected and actual results.

Do not move ahead until: Any double-counted stock, unapproved write or duplicate action is a release blocker.

Step 13. Run in shadow mode

Purpose: Compare proposed recommendations with normal staff decisions without changing stock.

What to do:

- Run the engine for the selected stores/SKUs while the current process remains in control.
- Have Operations review the suggestions and record whether they would act, change the amount or ignore it.
- Compare suggestions with real subsequent sales, stockouts, receipts, delays and expiry outcomes.
- Investigate repeated false alarms, excessive proposed quantities and missed urgent items.
- Agree a pilot threshold only after baseline costs and operational impact have been measured.

Expected output: A shadow-pilot report with examples of good and bad recommendations.

Do not move ahead until: Do not go live based only on forecast error. Review inventory outcomes and staff workload too.

Step 14. Deploy a small controlled pilot

Purpose: Introduce the recommendation as a controlled operational aid.

What to do:

- Choose a small number of stores and SKUs with reliable stock and order records.
- Publish a run schedule, named support contact and clear manual fallback.
- Keep all recommendations approval-based and display stale-data warnings.
- Review performance daily during the first agreed period; pause the engine if source data stop updating.
- Expand only after Operations accepts the measured results and support workload.

Expected output: Pilot sign-off, monitored schedule, rollback instructions and handover record.

Do not move ahead until: No expansion if stock accuracy, staff workflow or the integration remains unstable.

Step 15. Operate and improve the engine

Purpose: Maintain accuracy and prevent silent failures after launch.

What to do:

- Monitor data freshness, failed loads, missing records and source-to-warehouse count differences.
- Monitor forecast error, persistent over-forecasting/under-forecasting and the number of recommendations overridden.
- Measure stockouts, lost sales/cancellations where visible, excess stock, expiry/write-off, lead-time error and recommendation acceptance.
- Review performance by product group and location; retrain only after evaluation on a newer historical period.
- Keep the previous approved model and a manual rule-based fallback available for rollback.

Expected output: A monthly operating review and controlled model-update record.

Do not move ahead until: A model update must pass the same evaluation and approval gate as the original release.

8. How the forecast and stock recommendation are chosen

8.1 Keep forecast choice separate from stock policy

The forecast estimates future demand. The inventory planner decides what to do with that estimate. A forecast may be reasonably accurate yet still recommend the wrong action if reservations, arrivals, units, expiry or supplier rules are wrong. Test these two parts separately.

Candidate	What it means in plain language	Use it when	Caution
Recent average	Assume upcoming demand resembles a recent average.	A simple benchmark and short history.	Can miss a weekly pattern or sudden change.
Same weekday / same hour	Use a comparable past period, such as the previous Friday evening.	There is enough repeated calendar history.	Promotions and past stockouts can make the comparison unfair.
Smoothing method	Give newer observations more influence than older ones.	Demand changes gradually.	Does not automatically understand promotions or store differences.
LightGBM / XGBoost	Learn from past sales together with useful product, store, hour and calendar information.	There are reliable examples and the team can maintain the model.	Needs careful time-based testing and meaningful features.
TimesFM 3.0	A pretrained forecasting model designed for time-series patterns.	An approved commercial use route and local tests make it appropriate.	Do not self-host the publicly downloadable 3.0 weights for commercial production without permission.

8.2 Measures that matter

Measure	Plain-language meaning	How to use it
MAE	Average size of the forecast error in units.	Useful for explaining typical missed units, but high-volume products dominate.
WAPE	Total absolute forecast error compared with total actual demand.	Useful for overall business comparison; always show product/store breakdowns.
Bias	Whether the system usually predicts too much or too little.	Persistent positive bias can create excess stock; negative bias can increase stockouts.

Measure	Plain-language meaning	How to use it
Stockout rate / time out of stock	How often and how long the item could not be sold.	Evaluate after the inventory planner is simulated or piloted.
Excess / expiry / write-off	Stock that was not needed or could not be sold in time.	Especially important for perishables and short-shelf-life goods.
Recommendation acceptance and override	How often staff accept or change the recommendation.	Record reasons; repeated overrides may signal bad inputs or bad policy.
Lead-time error	Difference between expected and actual supply arrival.	Helps decide whether a conservative buffer or a later lead-time model is useful.

9. Worked example: one SKU at one store

Illustrative numbers only. These are not Accesco operating data or approved settings.

Input	Example value	How it is used
Usable stock on hand	24 units	Starting quantity available for sale.
Reserved for customers	5 units	Already committed, so not free to sell again.
Confirmed inbound before the target period	0 units	No incoming units can be counted yet.
Forecast daily demand	6 units/day	Used only because this example assumes daily data are reliable.
Expected supply lead time	3 days	Expected demand before more stock can arrive is about 18 units.
Extra review period	1 day	The target covers another 6 units before the next review/action.
Safety buffer	10 units	An Operations-approved example allowance for unexpected demand/delay.
Minimum order / case pack	12 units / 6 units	Recommendation must meet minimum and use whole case multiples.
Storage and shelf-life cap	At least 18 units	No cap prevents the calculated proposal in this example.

Calculation in plain words: usable inventory position starts at 24 on hand minus 5 reserved, which leaves 19 units. Target cover is 18 units during the three-day lead time, plus 6 units for the next review period, plus a 10-unit safety buffer, for a target of 34. The gap is 15 units. The 12-unit minimum is met, then the proposal is rounded up to a case multiple of 6, giving 18 units.

Example output: Propose 18 units for staff review. The recommendation must also show its input timestamps and rule settings. If actual lead time, reservations, pack size or expiry data are unknown, the real engine must flag the case instead of using these illustrative numbers.

10. Testing plan and acceptance checks

Test stage	What to test	Evidence required to pass
Data audit	Missing SKU/location IDs, duplicate order lines, impossible dates, unit mismatches, stock snapshots and history gaps.	Quality report approved by data owner; all critical failures fixed or clearly block affected SKUs.
History reconciliation	Compare selected SKU/store/day totals with source OMS and inventory reports.	Differences explained; no unexplained material mismatch.

Test stage	What to test	Evidence required to pass
Forecast backtest	Compare each candidate to baselines on later periods not used for training.	Results by node/category and clear error/bias report; chosen candidate provides a defensible benefit.
Planner scenarios	No stock; low stock; incoming delivery before/after predicted depletion; reserved stock; pack size; minimum order; expiry; capacity conflict.	Expected quantity and priority agree with documented inventory policy.
System integration	Slow API, unavailable source, duplicate event, partial extract, expired authentication and retry.	No duplicate actions; failed or stale data are visible; retry is safe.
Permission test	Read-only forecast service, approver role, blocked user, secret access and audit.	No unauthorized stock write or approval.
Shadow run	Compare proposals with real outcomes and staff judgements.	Operations signs off on forecast usefulness, rules and workload before live pilot.
Rollback test	Disable forecast, revert to previous approved version and use manual process.	Staff can continue safely without the engine.

Release blockers

- Any recommendation calculated from stale or missing critical stock.
- Any duplicate transfer or purchase request created after a retry.
- Any silent disagreement between SKU, pack unit or location identifiers.
- Any unapproved change to operational stock or order state.
- No reproducible backtest when the team claims a trained forecast has been validated.
- No clear manual fallback, monitoring owner or support process.

11. Pilot, deployment, rollback and ongoing support

Stage	What happens	Move to next stage when
Development	Use masked/controlled data or synthetic data to validate the pipeline and business rules.	The pipeline repeats, calculations are traceable and data quality is visible.
Offline evaluation	Test historical forecasts against later actuals; simulate replenishment decisions.	Forecast and inventory results meet thresholds agreed by Operations.
Shadow mode	Generate recommendations but do not submit stock actions.	Staff review results and key error patterns have been addressed.
Small pilot	Selected SKUs and locations; authorised approval remains mandatory.	Stable integration, manageable overrides, no serious stock or duplicate action issues.
Wider rollout	Expand in controlled groups and keep rollback available.	Every new site passes data-readiness checks and its policies are configured.
Routine operations	Schedule, monitor, review and update.	Each update uses the same controlled evaluation process.

Rollback plan

- Pause new recommendations or stop the scheduled run.
- Keep previous recommendations visible for audit but mark them expired if stock has changed.
- Return staff to the documented manual replenishment process.
- Restore the previous approved model or baseline if a model update caused the issue.
- Prevent the failed or paused service from sending operational write requests.
- Reconcile pending recommendations and duplicate-submission keys before restarting.

12. Monitoring, security and business measures

What to watch	Examples of warning signs	Action
Freshness	Inventory or receipt information has not updated within the agreed window.	Pause recommendations for affected locations; alert source owner.
Data health	Missing SKUs, unit mismatches, duplicate or rejected records increase.	Stop affected pipeline stage and correct mapping before scoring.
Forecast behavior	Error or bias grows, or a category regularly receives too-high suggestions.	Investigate calendar changes, stockouts and promotion inputs; compare candidates again.
Inventory outcomes	Stockout time, expiry/write-off or excess inventory rises.	Review the planner constraints, lead time and business targets, not only the model.
Integration	Timeouts, stale records or repeat submission attempts rise.	Retry safely, reconcile using unique request references and keep actions approval-based.
Security	Unexpected access or secret exposure.	Revoke access, preserve audit logs and follow incident procedure.

Minimum security controls

- Use read-only access during data collection; separate the identity used to generate recommendations from the identity allowed to submit approved requests.
- Keep credentials in the approved secret manager and rotate them according to policy.
- Limit access to raw order, customer and inventory history; show only operationally necessary fields in the review screen.
- Log the source-data cut-off, forecast version, calculation rules, approver, override reason and final transaction reference.
- Use separate development, test and production workspaces or access boundaries; do not experiment against the live stock ledger.
- Agree data retention, back-up and incident responsibilities with system owners.

13. Main risks and decisions required

Risk / open question	Why it matters	Mitigation / owner to confirm
Not enough sales history	Cannot show demand accuracy or build reliable time-series labels.	Measure history first; operate rule prototype only until ready. Data owner TBC.
Stockouts hide demand	Zero recorded sales may mean no stock, not no demand.	Include availability history and flag affected periods. IMS/WMS owner TBC.
Unclear node structure	Central warehouse stock may be confused with dark-store stock.	Define node type and allowed replenishment route. Operations owner TBC.
Unapproved TimesFM licensing	Self-hosted TimesFM 3.0 weights have restrictions.	Use an authorised commercial route or select a model with suitable rights. Technical/legal approver TBC.
System ownership conflicts	Some documents use IMS for different concepts or assign the same data to multiple systems.	Approve one system-of-record map and publish API/event contracts. System owners TBC.
No purchasing integration confirmed	An inventory recommendation is not automatically a supplier order.	Use a manual or draft proposal until SRM/procurement ownership is confirmed.
Expiry / unit gaps	A numerically correct order may still be unsafe or wasteful.	Require pack, unit and expiry validation; flag conflicts for staff review.
No named project team	No person can approve schema, policy or release choices.	Assign sponsor, technical lead, Operations lead and system owners before build.

14. Final go-live checklist

- [ ] Pilot scope, node types, time bucket and approving role agreed.
- [ ] SKU, location, order, inventory and receipt mappings documented.
- [ ] Actual fulfilled demand and availability history reconciled to source reports.
- [ ] Data quality checks and stale-data behaviour are working.
- [ ] Simple baselines compared fairly with any ML candidate.
- [ ] Forecast choice approved, including license route if TimesFM 3.0 is used.
- [ ] Inventory rules for lead time, safety buffer, pack size, minimum order, expiry and capacity approved by Operations.
- [ ] Recommendation reasons, timestamps, versions and staff decisions are auditable.
- [ ] API contracts and ownership of OMS/IMS/WMS/TMS facts approved.
- [ ] Retry, duplicate-submission, permissions and rollback tests passed.
- [ ] Shadow-pilot report accepted; a named support owner and manual fallback are in place.
- [ ] Any expansion or automatic action remains separately approved.

References and supporting documents

- Accesco, Dual Replenishment Engine Model Architecture (working draft, October 2026) - basis for warehouse forecasting plus a deterministic inventory planner.
- Accesco, Unified Admin Panel Specification (internal working document) - proposed system ownership: OMS order history, inventory IMS catalogue and stock, WMS physical fulfilment, TMS transport and delivery.
- Accesco, SRM_IMS_WMS_TMS_Integration.pdf (May 2026) - receipt, ETA, lead-time and supplier hand-off concepts; confirm that the described SRM and TMS capabilities exist in the chosen build.
- Google Research TimesFM repository: https://github.com/google-research/timesfm - verify the exact model version and commercial/production license before use.
- XGBoost official documentation: https://xgboost.readthedocs.io/ - reference only if XGBoost/LightGBM is selected for the warehouse model.
- Databricks model lifecycle documentation: https://docs.databricks.com/aws/en/machine-learning/manage-model-lifecycle - confirm available features in the contracted workspace.
- All source documents are working inputs. The exact production APIs, schemas, event names and system owners remain to be confirmed by Accesco Engineering.

Implementation readiness sign-off

Complete this register during kickoff and before each release gate. Roles are listed, but all individual owners remain yet to be assigned.

Decision / approval	Approver role	Status	Date / reference
Pilot node types, locations and SKU scope	Operations owner	Yet to be assigned	TBC
Authoritative OMS / IMS / WMS / TMS data map	Owners of each system	Yet to be assigned	TBC
Demand history and stockout/availability data quality	Data / ML lead + source owners	Yet to be assigned	TBC
Inventory policy: service target, buffer, pack, MOQ, expiry and capacity	Inventory / Operations owner	Yet to be assigned	TBC
Forecast candidate and commercial use permission	Technical owner + legal/procurement if required	Yet to be assigned	TBC
API/event contracts and safe retry behavior	Backend/integration owner + system owners	Yet to be assigned	TBC
Shadow pilot acceptance and business measures	Operations owner + sponsor	Yet to be assigned	TBC
Production deployment and rollback approval	Sponsor + technical owner	Yet to be assigned	TBC